Distributed Transaction

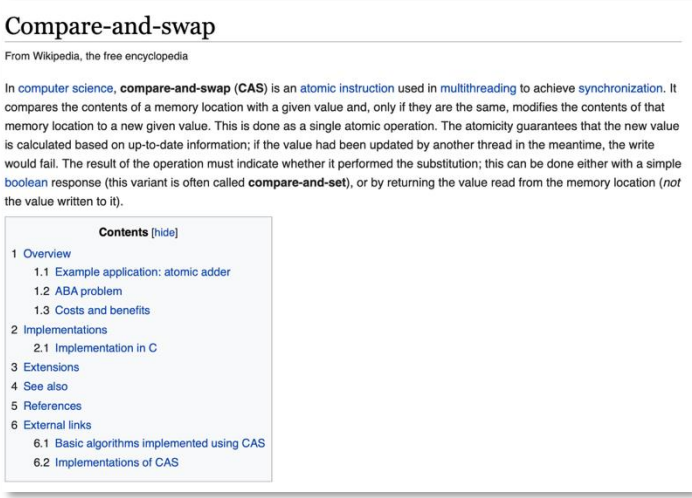
CUHK

Transactions and ACID in (single-node) DB

- T
  - Withdraw $100 from Account A
  - Deposit $100 to Account B
- **A**tomicity: All or Nothing
- **C**onsistency: C in single-node DB != C in distributed world
  - End Result = preserve the invariants (e.g., constraint: account >=0)
- **I**solation: Multiple Concurrent Transactions T1 and T2
  - End Result = **Serializable** = T1 T2 or T2 T1 but not something else
  - Concurrency Control (e.g., locking)
- **D**urability: Result of a committed transaction persist no matter what
  - e.g., earthquake, machine failure
  - Logging and Recovery (very different from HA from Distributed World)

Concurrency control (OS vs DB) on single node

- OS:
  - Concurrent R/W to a **single-value**
  - Correctness: **linearizability**
  - **Solution:**
    - Locking (Semaphore; Pthread lock)
    - Lock-free (Use of CAS)
- DB:
  - A **transaction** involves R/W on **multiple values**
  - Concurrent transactions
  - Correctness: **ACID**
  - **Solution:**
    - Pessimistic
    - Optimistic



Compare-and-swap

From Wikipedia, the free encyclopedia

In computer science, **compare-and-swap** (**CAS**) is an atomic instruction used in multithreading to achieve synchronization. It compares the contents of a memory location with a given value and, only if they are the same, modifies the contents of that memory location to a new given value. This is done as a single atomic operation. The atomicity guarantees that the new value is calculated based on up-to-date information; if the value had been updated by another thread in the meantime, the write would fail. The result of the operation must indicate whether it performed the substitution; this can be done either with a simple boolean response (this variant is often called **compare-and-set**), or by returning the value read from the memory location (*not* the value written to it).

Contents [hide]

1 Overview
  1.1 Example application: atomic adder
  1.2 ABA problem
  1.3 Costs and benefits
2 Implementations
  2.1 Implementation in C
3 Extensions
4 See also
5 References
6 External links
  6.1 Basic algorithms implemented using CAS
  6.2 Implementations of CAS

Transactions on single-node DB

- Pessimistic
  - 2 Phase Locking (2PL)
    - As long as a txn wants to access (read/write) an item, it must acquire a lock of that item first
    - A txn that touches multiple items
      - once unlock //then step into phase 2
        - **you can't acquire any lock again***
    - 2PL guarantees you a **serializable** schedule = the order of the final lock acquired
    - Good: little coordination between transactions
      - Each transaction locally respects 2PL protocol and accesses the central lock table
    - Bad: deadlock (will abort one), lock overhead
  - Fine-grained locks: reader-writer

	S	X
S	✔	✗
X	✗	✗

- Optimistic
  - Instead of waiting for the lock
    - let them interleave whatever, but **abort** on observing conflicts and **restart**

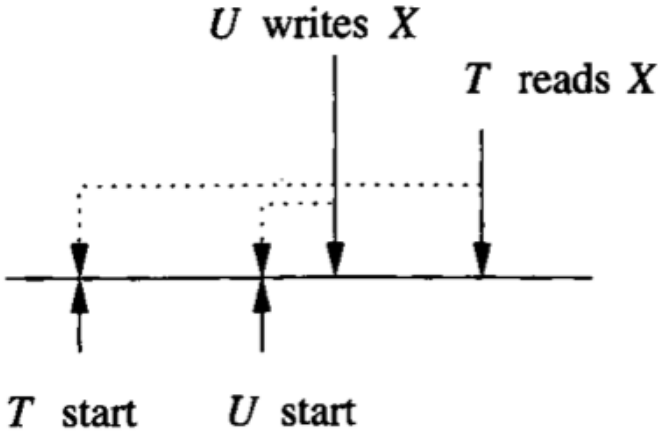
*In practice, for A and D, the locks are released after writing a 'commit request' to the log buffer, namely Strict 2PL

Transactions on single-node DB

- Optimistic
  - **Timestamp-ordering**
    - Cross check between timestamps of txns on data items **during read and write**
    - The equivalent serializable order = timestamp order of the transactions
    - E.g.,
      - Serializable order is: T < U
      - Whenever you (T) **read** X
        - Last-time-X-**written**-by-others (e.g., U)

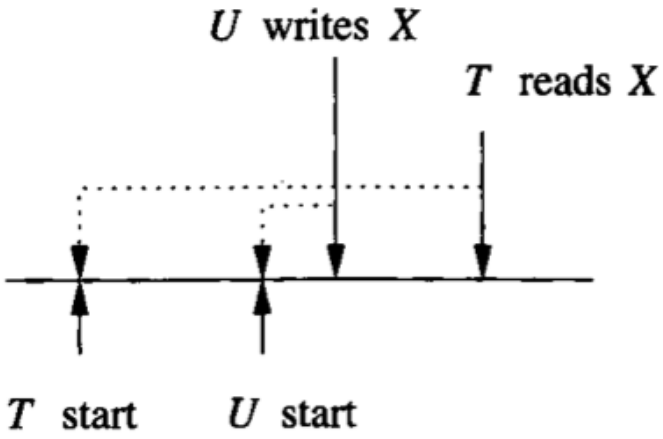
          >

          Your start timestamp (i.e., T start)
        - ”Read too late” ➔ Abort and Restart T

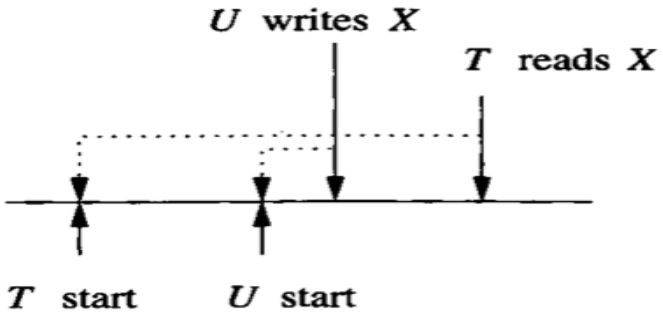


Transactions on single-node DB

- Optimistic
  - **Multi-version Timestamp Ordering** (MVTO)
    - If keeping **versions** of X
      - Then T can read the previous version of X
    - Tradeoff is?



Transactions on single-node DB



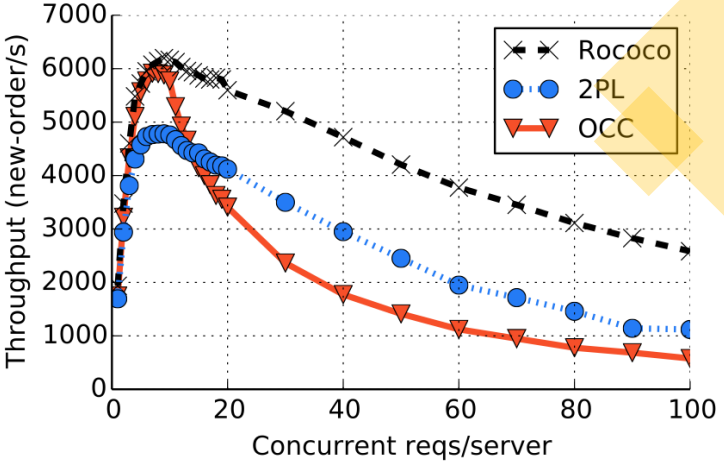
- Optimistic
  - **Validation based** (a.k.a. OCC*)
    - Space overhead spent on **active transactions only**
  - Execution Phase 1:
    - The writes are written into **thread-local in-memory write set** (not to DB yet)
  - When a transaction T is ending
    - Validation Phase 2:
      - Validate its read-set and write-set with other concurrent transactions who have already committed
    - Commit/Rollback Phase 3
      - If no overlap / no serializability conflict / no problem after reordering
        - Lock, apply the write-set, unlock
      - Else
        - Abort T and Restart T

Advantages:
1) lock span is << 2PL
2) locks only when no conflict

*very confusing naming convention; But we follow Hector's Complete book convention here

Pessimistic vs Optimistic

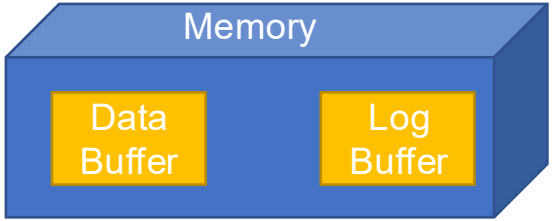
- Locking
  - Setting a lock on/off = ***writing shared memory*** 0/1 = overhead
  - If low-contention
    - (e.g., T1 writes X but T2 reads Y)
    - Locking overhead is for nothing
    - → hurt throughput
- Optimistic
  - If low-contention
    - → little overhead → better throughput
  - If high-contention
    - → Abort&Restart&Abort&Restart.. → waste of work



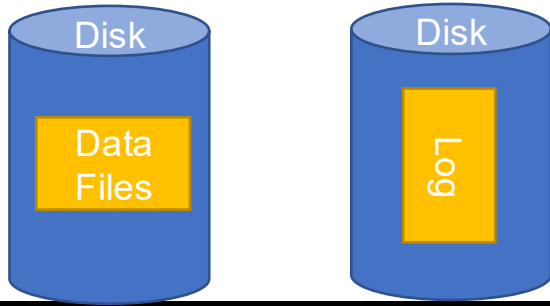
(a) Throughput

ACID

- Data
  - The unit is “page”, e.g., 8KB
    - You update 1 item, you update a page
  - Hot page in **data buffer** and full data (index/row/column) on **data disk**
    - Except for MVCC, otherwise in-place update
  - Data buffer full / Checkpointing? Flush updates to data disk
    - May flush updates from in-flight transactions to disk (”steal” policy)



- WAL (Write-Ahead Logging)
  - Any write to the **data disk** MUST write the update log **<u>to the log disk first</u>**
    - <tid, b4 value, after value> //to log buffer
    - <tid, COMMIT> //to log disk
  - Write to data disk vs log disk: random vs sequential write

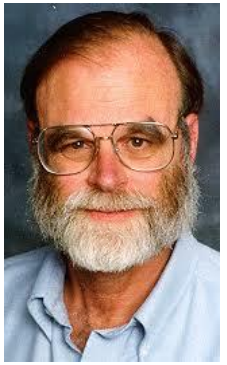


ACID

- On txn T commit: must flush log entries of T to **log disk (hence committing = flushing log buffer)**
  - Note: Updates on the **data** may NOT flush to the **data disk** even it is committed
    - Keep hot items in data buffer is fine as log is persisted (“no-force” policy)
- Crash and recovery:
  - UNDO incomplete transactions for persisted partial updates
  - REDO completed transactions for un-flushed updates

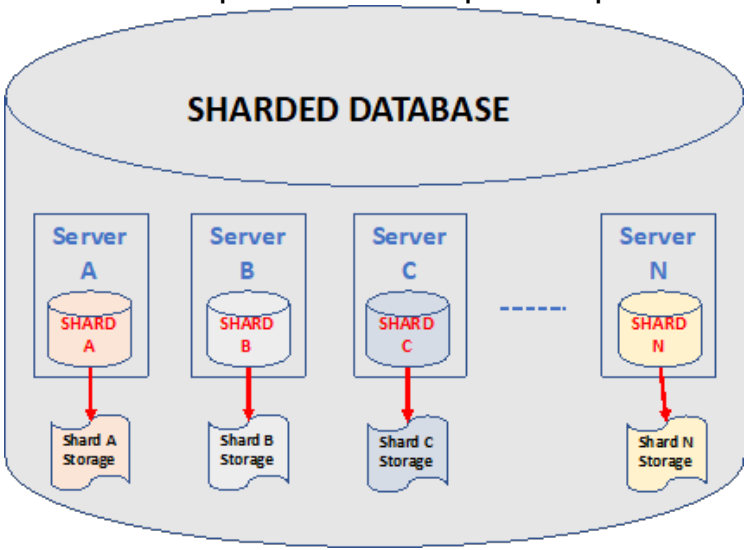
A. Begin
B. Growing phase
    i. Acquire locks
C. Reads, Computations, Writes (DRAM / flushed)
D. Commit Request
    i. Write commit to log buffer
E. Release Locks
F. Commit
    i. Flush log buffer to disk
G. Return to user

Multi-node Transactions and *A*CID



Reason 1: Naturally distributed

Reason 2: Performance (scale-out to multiple servers for parallel processing)

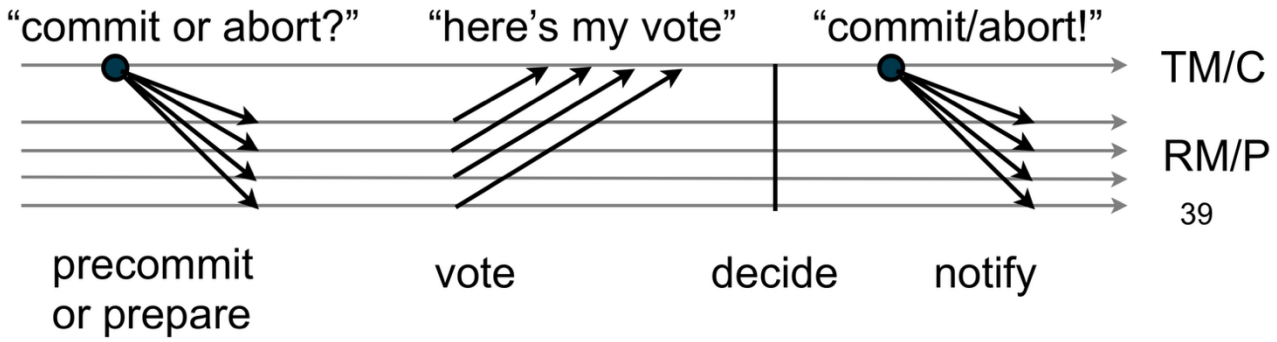


[src: oracle]

2PC (prepare/voting phase + commit/completion phase)

- TM = tran manager (coordinator)
- RM = remote manager (participant)

If unanimous to commit
decide to commit
else decide to abort



RMs validate Tx and prepare by logging their local updates and decisions

TM logs commit/abort (commit point)

29

Might block forever if the coordinator (a.k.a. transaction manager) fails or disconnects

https://www2.cs.duke.edu/courses/fall07/cps212/consensus.pdf

Putting everything together: CC + WAL + 2PC

	Coordinator shard //a is on this site	Participant shard //b is on this site

Execution	sLock(a)
	Read(a)
	Remote.sLock+Read(b)
		sLock(b) //may delay because of other txn here
		Return b;

	a = a - 10 b = b + 10 //not write to disk yet!	

2PC-prepare	Coordinator.Prepare(T, a), Remote.Prepare(T, b)
	xLock(a), WAL(a) //on recv <prepare> msg
	Return <Prepared> / <Failed> (if deadlock)
		xLock(b), WAL(b) //on recv <prepare> msg
		Return <Prepared> / <Failed> (if deadlock)

2PC-commit	If all shards s say "Prepared" (or 1 of them is aborted) WAL(T is committed/aborted); others.Commit/Abort(T); unlock(a)	WAL(T is committed/aborted) unlock(b)

Summary

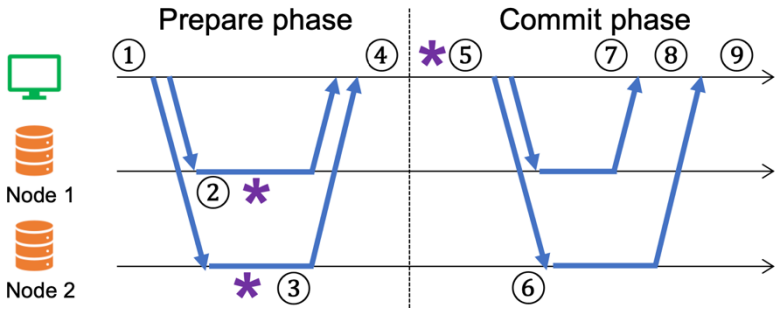
- Linearizability (Replicated State Machine, Distributed):
  - Multiple replicas on a **“single” logical object (e.g., a log, a kv store)**
    - But behaves like a single-threaded machine
- Serializability (Shared Memory, Single-Node DB):
  - Multiple threads (transactions) on **multiple items**
    - But it must behave like **any** serial order
      - So: T1 T2 T3 or T2 T1 T3 is also fine

http://www.bailis.org/blog/linearizability-versus-serializability/

What is the correctness of transaction + sharding + replication?

- Test your creativity
  - Distributed transaction without replication, i.e., sharding only?
    - (Global) Serializability
  - Distributed transaction with replication?
    - One-copy serializable (1SR)

9 cases of fail-stop during 2PC



(1) before the coordinator sends prepare requests

(2) after some participant nodes receive prepared requests

(3) after all participant nodes receive prepared requests

(4) before the coordinator receives all votes from participant nodes

(5) after the coordinator writes the commit record

(6) before some participant nodes receive commit requests

(7) before the coordinator receives any acknowledgement

(8) after the coordinator receives some acknowledgements, and

(9) after the coordinator receives all acknowledgements.

Lu et al. VLDB 2021

Availability angle: 2PC is not fault-tolerant

- All processes agree on {Commit, Abort}
- To counter FLP
  - We need safety more than liveness in this $ case
- 2-Phase Commit
  - If the coordinator crashes after some participants are prepared, those participants may not know whether to commit or abort, so they block.
- 3-Phase Commit
  - 3 Phases: can-commit (prepare), pre-commit (prepare-to-commit), do-commit (commit)
    - If the coordinator fails, participants can *decentralize* and communicate among themselves; if they find that all were in the pre-commit state, they can safely commit; otherwise they must abort.
    - Eliminated some (coordinator crashes case) but not all blocking cases
- Can Paxos solve atomic commit? Yes
  - Paxos Commit [Gray and Lamport; 2x Turing Award; 2004]
    - Give 2F+1 replications to coordinators/participants to make them fault-tolerant
    - Less efficient than 2PC (which is fair because it is more fault-tolerance)
    - With 2F+1 coordinators, hence, non-blocking up to F failures.

Scalability angle: 2PC is a bottleneck of distributed DB

- Locks must be held until 2PC is ending
- Significantly hurt concurrency ➔ hurt throughput/latency/scalability

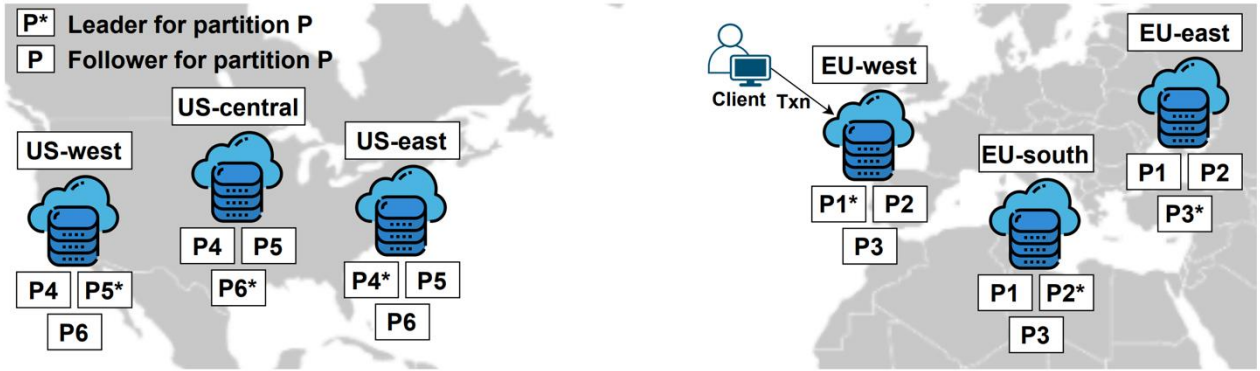


Figure 4: An example geo-distributed database. A data partition P is replicated to several data centers. P^* stands for the the leader replica.

The history	SQL DB ~1980	Distributed DB <2000	NoSQL 2006	NewSQL1 2008	NewSQL2 2012	From 2012 to 2023
(Multi-Item) Txn	Yes	Yes	NO	Yes	Yes	Yes
Sharding	-	2PC	Yes, but no 2PC is needed	Yes, but no 2PC is needed because assume no cross-shard txn	Yes, support cross-shard txn using 2PC	How can we combine atomic commit + replication such that…
Replication	-	-	Yes	-	Yes, Paxos	
Scale-out (key focus)	**-**	**V. bad when dist txn**	**Yes (as no 2PC)**	**Yes (as no 2PC)**	**Not good when dist txn**	**… they scale better here when facing dist txn?**
Correctness: txn level + replication level	Serializable + {}	(Global) Serializable + {}	{} + Eventual/ Causal	(Global) Serializable + {}	External Consistency = 1-copy serializable + linearizability (serial order follows real-time) (when no P)	One-copy Serializable (when no P)
Example	MySQL	MySQL Cluster	BigTable	H-Store	Spanner CockroachDB	MDCC -> TAPIR -> GPAC -> Primo -> Bonspiel
Anecdote			"SQL is dead…"	"*A major step backward*", M. Stonebraker	Google: "we need transactions…" CockroachDB 278m USD	

Geo-distributed Database Product

- Google Cloud Spanner (no P under CAP?)
- Amazon Aurora Global Database
- Microsoft Azure Cosmo DB
- Alibaba PolarDB-X

ockroach Labs is generating substantial revenue, estimated at approximately **$128.3 illion in 2023**. While the private company does not publicly disclose its net profit or oss, it has entered a "high-growth" phase typically focused on scaling Annual Recurring evenue (ARR) rather than immediate profitability. CNBC +3

ey financial highlights include:

- **Revenue Growth**: The company reached roughly $128.3 million in annual revenue by late 2023, a significant jump from an estimated $30 million in 2021.
- **Business Model**: They make money through two primary streams:
  - **Managed Cloud (SaaS)**: A fully managed "database-as-a-service" which is their fastest-growing segment.
  - **Enterprise Self-Hosted**: Selling premium features like 24/7 support and advanced

Cockroach Labs raised **$6.25 million in seed funding in June 2015** to develop its distributed SQL database, backed by major investors including Benchmark, Google Ventures, and Sequoia Capital. Founded by former Google engineers, the company has since grown to a $5 billion valuation with over $633 million in total funding as of late 2021. Reuters +4